

CAPITULO ESTUDIANTEL ACM - ICPC UMSA

1

CAMPAMENTO DE PROGRAMACIÓN COMPETITIVA

ENERO 2019





Edson Eddy Lecoña Zarate
Oscar Antonio Rondo Rodriguez

Carrera de Informatica
Facultad de Ciencias Puras y Naturales



Contenido

1 C++

Tipos de datos C++
Entrada/Salida C++

2 STL C++

Vector
Stack
Queue
Deque
Priority Queue (Cola de prioridad)
Set
Map
Iterator

3 Recursividad

4 Fuerza Bruta



1

C++



UMSA
la mejor

Tipos de datos C++



Los tipos de datos que maneja C++ son:

Lógico: bool.

Enteros signo: short, int, long, long long.

Enteros sin signo: unsigned short, unsigned int, unsigned long.

Números en coma flotante: float, double, long double.

Carácter: char.

Cadena: string.



Tipos de datos C++



Type	Bytes	Min value	Max value
bool	1	0	1
char	1	-128	127
short	2	-32768	32767
int	4	-2148364748	2147483647
long long	8	-9223372036854775808	9223372036854775807
	n	-2^{8n-1}	$2^{8n-1} - 1$

Type	Bytes	Min value	Max value
unsigned char	1	0	255
unsigned short	2	0	65535
unsigned int	4	0	4294967295
unsigned long long	8	0	18446744073709551615
	n	0	$2^{8n} - 1$

Type	Bytes	Min value	Max value	Precision
float	4	$\approx -3,4 \times 10^{38}$	$\approx 3,4 \times 10^{38}$	≈ 7 digits
double	8	$\approx -1,7 \times 10^{308}$	$\approx 1,7 \times 10^{308}$	≈ 14 digits
long double	16	$\approx -1,1 \times 10^{4932}$	$\approx 1,1 \times 10^{4932}$	≈ 18 digits





La librería **iostream** es un componente de la biblioteca estándar de C++ que es utilizado en operaciones de entrada/salida.

```
#include <iostream>
```

Entrada

El comando **cin** es para la introducción de datos por teclado, este comando esta seguido por el operador >>, a continuación la variable.

```
TipoDato nameVar;  
cin>>nameVar;
```





Salida

El comando **cout** es para la impresión de datos por consola, este comando esta seguido por el operador <<, a continuación la variable.

```
TipoDato nameVar;  
cout<<nameVar;
```

El comando **endl** indica salto de linea.

```
TipoDato nameVar;  
cout<<nameVar<<endl;
```



Entrada/Salida C++ (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 using namespace std;
3 int main(){
4     int n;           //inicializando variable
5     cin >> n;         //lectura de la variable
6     cout << n << endl; //impresión de la variable
7     return 0;
8 }
```



2

STLC++



UMSA
la mejor



La STL (La Standard Template Library) es una colección de estructuras de datos y algoritmos de uso común. La STL se podría dividir en tres grandes partes: Contenedores (plantillas de estructuras de datos populares), iteradores y algoritmos.

Los **Contenedores** de la STL son estructuras de datos capaces de contener casi cualquier tipo de objeto.

Generalizando se tiene:

```
TipoEstructura <TipoDato> name;
```



```
vector<tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica lineal.
- Es un contenedor secuencial que representa un Arreglo.

Algunos métodos:

push_back(dato)	Insertar un dato al final del Vector.
size()	Retorna la longitud del Vector.
empty()	Retorna true si el Vector está vacío, caso contrario false.
clear()	Elimina todos los datos del Vector.



Vector (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <vector> //importando libreria vector
3 using namespace std;
4 int main(){
5     int n, x;
6     cin >> n;
7     vector <int> V; // vector<int> V(n) vector estático
8     for (int i = 0; i < n; i++){ //lectura de datos
9         cin >> x;
10        V.push_back(x); //insertando elemento al vector
11    }
12    for (int i = 0; i < n; i++){
13        cout << V[i] << " " << endl; //impresion de datos
14    }
15 }
```



Stack (Pila)



```
stack <tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica lineal.
- Sigue el principio de LIFO (Last In First Out).

Algunos métodos:

push(dato)	Introduce un dato en la Pila.
top()	Accede al último elemento de la Pila.
pop()	Elimina el último elemento de la Pila.
size()	Retorna el tamaño de la Pila.
empty()	Retorna true si la Pila está vacía, caso contrario false.



Stack (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <stack> //importando libreria stack
3 using namespace std;
4 int main(){
5     int n, x; cin >> n;
6     stack <int> S; //inicializando pila
7     for (int i = 0; i < n; ++i){
8         cin >> x; S.push(x); //insertando elemento en la pila
9     }
10    while(!S.empty()){
11        x = S.top(); //extrayendo un elemento de la pila
12        S.pop(); //eliminando un elemento de la pila
13        cout << x << endl; //impresion del elemento
14    }
15    return 0;
16 }
```



Queue (Cola)



```
queue <tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica lineal.
- Sigue el principio de FIFO (First In First Out).

Algunos métodos:

push(dato)	Introduce un dato en la Cola.
front()	Accede al primer elemento de la Cola.
pop()	Elimina el primer elemento de la Cola.
size()	Retorna el tamaño de la Cola.
empty()	Retorna true si la Cola está vacía, caso contrario false.



Queue (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <queue> //importando libreria queue
3 using namespace std;
4 int main(){
5     int n, x; cin >> n;
6     queue <int> S; //inicializando Cola
7     for (int i = 0; i < n; ++i){
8         cin >> x; S.push(x); //insertando elemento en la cola
9     }
10    while(!S.empty()){
11        x = S.front(); //extrayendo elemento de la cola
12        S.pop(); //eliminando elemento de la cola
13        cout << x << endl; //mostrando elemento de la cola
14    }
15    return 0;
16 }
```



Deque (Bicola)



```
deque <tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica lineal.
- Es una variación de Pila y Cola.

Algunos métodos:

push_back(dato)	Introduce por detrás un dato a la bicola.
push_front(dato)	Introduce por adelante un dato a la bicola.
front()	Accede al primer elemento de la bicola
back()	Accede al último elemento de la bicola
pop_back()	Elimina el último dato de la bicola.
pop_front()	Elimina el primer dato de la bicola.
size()	Retorna el tamaño de la bicola.
empty()	Retorna true si la bicola está vacía, caso contrario false.



Deque (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <deque> //importando libreria deque
3 using namespace std;
4 int main(){
5     int n, x; cin >> n;
6     deque <int> S; //inicializando la cola
7     for (int i = 0; i < n; ++i){
8         cin>>x; S.push_front(x); //insertando elemento al comienzo
           de la cola
9     }
10    while(!S.empty()){
11        x = S.front(); //extrayendo elemento de la cola
12        S.pop_front(); //eliminando elemento de la cola
13        cout << x << endl; //mostrando elemento de la cola
14    }
15    return 0;
16 }
```



Priority Queue



```
priority_queue <tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica no lineal.
- Es una variación de Cola.
- Los elementos ingresados se ordenan automáticamente (Según Prioridad).

Algunos métodos:

push(dato)	Introduce un dato en la Cola.
top()	Accede al primer elemento de la Cola.
pop()	Elimina el primer elemento de la Cola.
size()	Retorna el tamaño de la Cola.
empty()	Retorna true si la Cola está vacía, caso contrario false.



Priority Queue (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <queue> //importando libreria queue
3 using namespace std;
4 int main(){
5     int n, x; cin >> n;
6     priority_queue <int> S; //inicializando cola
7     for (int i = 0; i < n; ++i){
8         cin >> x; S.push(x); //insertando elemento en la cola
9     }
10    while(!S.empty()){
11        x = S.top(); //extrayendo elemento de la cola
12        S.pop(); //eliminando elemento de la cola
13        cout << x << endl; //mostrando elemento de la cola
14    }
15    return 0;
16 }
```



Set (Conjunto)



```
set <tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica no lineal.
- No admite elementos repetidos (elementos únicos).
- Sus funciones principales Búsqueda/Eliminación/Inserción.
- Los elementos ingresados se ordenan automáticamente.
- Solo se puede acceder a sus elementos mediante el uso de iteradores.

Algunos métodos:

insert(dato)	Introduce un dato en el Set.
count()	Consulta si un elemento se encuentra en el Set.
erase(dato)	Encuentra un dato y lo elimina.
clear()	Limpia el contenido del Set.
size()	Retorna el tamaño del Set.
empty()	Retorna true si el Set está vacío, caso contrario false.



Set (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <set> //importando libreria set
3 using namespace std;
4 int main(){
5     int n, x; cin >> n;
6     set <int> S; //inicializando set
7     for (int i = 0; i < n; ++i){
8         cin >> x; S.insert(x); //insertando elemento al set
9     }
10    cout<< S.count(1) << endl; //Consulta si 1 se encuentra en el
        set
11    cout<< S.count(3) << endl; //Consulta si 3 se encuentra en el
        set
12    cout<< S.count(5) << endl; //Consulta si 5 se encuentra en el
        set
13    return 0;
14 }
```



Map (Mapa)



```
map <tipoDato, tipoDato> nombre;
```

Características:

- Es una estructura de datos dinámica no lineal.
- Los elementos se almacenan en la estructura según Llave-Valor.
- No admite elementos repetidos (llave única).
- Sus funciones principales Búsqueda/Eliminación/Inserción.
- Los elementos ingresados se ordenan automáticamente (Según llave).

Algunos métodos:

[llave]	Accede un elementos del Map mediante su llave.
erase(dato)	Elimina el dato buscado si se encuentra en el Map.
size()	Retorna el tamaño del Map.
empty()	Retorna true si el Map está vacío, caso contrario false.



Map (Ejemplo)



```
1 #include <iostream> //importando libreria E/S
2 #include <map> //importando libreria map
3 using namespace std;
4 int main(){
5     int n, x; string st;
6     cin >> n;
7     map <string, int> S; //inicializando map, llave=string, valor
      =int
8     for (int i = 0; i < n; ++i){
9         cin >> x >> st;
10        S[st] = x; //asignando valor x a la llave st
11    }
12    cout<< S["abc"] << endl; //mostrando lo que contiene la llave
      abc
13    cout<< S["ab"] << endl; //mostrando lo que contiene la llave
      ab
14    cout<< S["a"] << endl; //mostrando lo que contiene la llave a
15    return 0;
```



Iterator (Iteradores)



```
tipoEstructura <TipoDato> :: iterator nombre;
```

Un **iterator** es un objeto que se mueve a través de un contenedor de otros objetos y selecciona a uno de ellos cada vez, sin proporcionar un acceso directo a la implementación del contenedor.

Los iteradores proporcionan una forma estándar de acceder a los elementos.

Algunos métodos:

.begin()	Retorna el iterador del primer elemento, si el está vacío retorna .end().
.end()	Retorna el iterador del elemento siguiente al último elemento



Iterator (Ejemplo)



```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     set <int> S; //inicializando Set
5     S.insert(12); //insertando elementos
6     S.insert(0);
7     S.insert(5);
8     set <int> :: iterator it; //inicializando iterador
9     for (it = S.begin(); it != S.end(); it++){
10         cout<<*it<<endl; //mostrando elemento que apunta el
            iterador
11     }
12     return 0;
13 }
```



3

Recursividad



UMSA
la mejor



Recursion es la forma en la que se especifica un proceso basado en su propia definición.

Cuando una función se llama a si misma se dice que es una función recursiva.





Para resolver un problema recursivamente, tenemos que estar seguros que:

- El problema puede ser dividido en subproblemas.
- El problema tiene algún caso base.
- La cantidad de llamadas recursivas no sobrecargara la pila de la función.





EJEMPLO 1:

para todo $x \geq 0$.

$$\text{Factorial}(x) = x * (x-1) * (x-2) * \dots * (1).$$

$$\text{Factorial}(x) = x * \text{Factorial}(x-1).$$

$$\text{Factorial}(5) = 5 * 4 * 3 * 2 * 1 = 120.$$

$$\text{Factorial}(0) = 1.$$



Recursividad



```
1  int factorial(int n){
2      if(n==0)                //Caso Base
3          return 1;
4      return n*factorial(n-1); //Recursion
5  }
```





EJEMPLO 2:

para todo $x \geq 0$.

$\text{fibonacci}(x) = \text{fibonacci}(x-1) + \text{fibonacci}(x-2)$.

$\text{fibonacci}(0) = 0$.

$\text{fibonacci}(1) = 1$.



Recursividad



```
1  int fibo(int n){  
2      if(n==0)                                //caso base  
3          return 0;  
4      if(n==1)                                //caso base  
5          return 1;  
6      return fibo(n-1) + fibo(n-2); //recursion  
7  }
```





EJEMPLO 3:

Hay n personas en una habitación. Si cada persona se da la mano **una vez** con el resto. ¿Cuál es el número total $h(n)$ de apretones de manos?



Recursividad



```
1  int saludo(int n){  
2      if(n<=1)                //caso base  
3          return 0;  
4      return (n-1)+saludo(n-1); //recursion  
5  }
```



4

Fuerza Bruta



UMSA
la mejor



Fuerza bruta o también conocido como búsqueda completa es una técnica en la que resolvemos un problema atravesando completamente el espacio de búsqueda para obtener la solución.

Mientras buscamos una solución, podemos eliminar partes del espacio de búsqueda si determinamos que la solución no se encuentra en esas partes.





EJEMPLO 1:

Contar todos los pares (A,B) que cumplan $A/B = 17$.

Donde $(1 \leq A,B \leq 1000)$.



Fuerza Bruta



```
1  int cd=0;
2  for (int A = 1; A <= 1000; ++A){
3      for (int B = 1; B <= 1000; ++B){
4          if(A%B == 0 && A/B == 17)
5              cd++;
6      }
7  }
```





EJEMPLO 2:

Contar todos los pares (A,B) que cumplan $A/B = 17$.

Donde $(1 \leq A,B \leq 1000000)$.



Fuerza Bruta



```
1  int cd=0;
2  int A;
3  for (int B = 1; B <= 1000000; ++B){
4      A=B*17;
5          if(A<=1000000)
6              cd++;
7  }
```





EJEMPLO 3:

De cuantas maneras se pueden colocar N reinas en un tablero de $N \times N$ de tal manera que no se ataquen entre si?

Donde $(1 \leq N \leq 10)$.



Fuerza Bruta



```
1  int table[11][11], N, cd=0;
2  bool EsSeguro(int x, int y){
3      for (int i = 0; i < 11; ++i){
4          if(table[x][i]==1)
5              return false;
6      }
7      for (int i = 0; i < 11; ++i){
8          for (int j = 0; j < 11; ++j){
9              if(table[i][j] == 1 && (i + j == x + y || i - j ==
10                 x - y))
11                  return false;
12          }
13      }
14      return true;
15  }
```



Fuerza Bruta



```
1 void backtrack(int col){
2     if(col >= N){                //caso base
3         cd++;
4         return;
5     }
6     for (int i = 0; i < N; ++i){
7         if(EsSeguro(i,col)){
8             table[i][col]=1;
9             backtrack(col+1);
10            table[i][col]=0; //backtrack
11        }
12    }
13 }
```



The background consists of several overlapping, three-dimensional rectangular blocks in various shades of red and pink. The blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be in front of others. The colors range from a deep, dark red to a very light, almost white pink.

¿Preguntas?



Gracias!